The Edward S. Rogers Sr. Department of

Electrical and Computer Engineering

University of Toronto

ECE496Y Design Project Course

Group Final Report

Project Title	Building an Intelligent Job Scheduler for Optimized Request Assignment
Team Number	2024105
Section Number	LEC 0101
Supervisor	Ali Bereyhi
Administrator	Sarah Oliveira
Team Member and Email Address	Yifan Wang (yfan.wang@mail.utoronto.ca) Housen Zhu (benjamin.zhu@mail.utoronto.ca) Ruoyu Li (lambert.li@mail.utoronto.ca) Yuan Sui (yuan.sui@mail.utoronto.ca)
Submission Date	3/21/2025

Group Final Report Attribution Table

This table should be filled out to accurately reflect who contributed to each section of the report and what they contributed. Provide a **column** for each student, a **row** for each major section of the report, and the appropriate codes (e.g. 'RD, MR') in each of the necessary **cells** in the table. You may expand the table, inserting rows as needed, but you should not require more than two pages. The original completed and signed form must be included in the hardcopies of the final report. Please make a copy of it for your own reference.

Section	**Student Names**			
	Housen Zhu	Ruoyu Li	Yuan Sui	Yifan Wang
Group Highlights	-	RD	ET	MR
Acknowledgements	RD	-	ET, MR	-
1. Executive Summary	MR	-	RD	ET
2. Table of Contents	-	RD, MR	-	-
3. Introduction	RS	MR	ET	RS, RD
4.1.1 Graphical User Interface Overview	-	MR, ET	-	RD, ET
4.1.2 Backend Overview	MR, ET	-	RD	ET
4.2 System Block Diagram	-	ET	MR	RD
4.3.1 Emergency Department Simulator Module	MR, ET	-	RD	-
4.3.2 DQN Model Design	RD	-	MR, ET	-
4.3.3 Deep Reinforcement Learning Training Technique	RD	-	MR, ET	-
4.3.4 Graphical User Interface Design	ET	RD	-	MR

4.3.5 Chatbot Design	ET	MR	-	RD
4.4.1 Effectiveness and Performance Analysis	RD	MR	-	ET
4.4.2 Limitations	-	-	RD	MR, ET
5.1 Training and Testing Dataset	MR, ET	-	RD	ET
5.2 DRL Model Test and Verification	RS, RD	-	MR, ET	-
5.3 Graphical User Interface Test and Verification	-	RS, RD	-	MR, ET
5.4 Chatbot Test and Verification	-	MR, ET	-	RS, RD
6. Summary and Conclusions	MR	-	RD	ET
7. Future Work	-	ET	MR	RS, RD
8. References	MR	RD	-	ET
9. Appendices	RD	-	-	MR
All	FP	FP	FP	FP

Abbreviation Codes:

Fill in abbreviations for roles for each of the required content elements. You do not have to fill in every cell. The "**All**" row refers to the complete report and should indicate who was responsible for the final compilation and final read through of the completed document.

RS – responsible for research of information
RD – wrote the first draft
MR – responsible for major revision
ET – edited for grammar, spelling, and expression
OR – other
"All" row abbreviations:

FP – final read through of complete document for flow and consistency
CM – responsible for compiling the elements into the complete document
OR - other

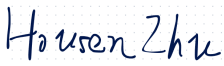
If you put OR (other) in a cell please put it in as OR1, OR2, etc. Explain briefly below the role referred to:

OR1: enter brief description here

OR2: enter brief description here

Signatures

By signing below, you verify that you have read the attribution table and agree that it accurately reflects your contribution to this document.

Name	**Yifan Wang**	**Signature**		**Date: 3/21/2025**
Name	**Housen Zhu**	**Signature**		**Date: 3/21/2025**
Name	**Ruoyu Li**	**Signature**		**Date: 3/21/2025**
Name	**Yuan Sui**	**Signature**		**Date: 3/21/2025**

Voluntary Document Release Consent Form

To all ECE496 students:

To better help future students, we would like to provide examples that are drawn from excerpts of past student reports. The examples will be used to illustrate general communication principles as well as how the document guidelines can be applied to a variety of categories of design projects (e.g. electronics, computer, software, networking, research).

Any material chosen for the examples will be altered so that all names are removed. In addition, where possible, much of the technical details will also be removed so that the structure or presentation style are highlighted rather than the original technical content. These examples will be made available to students on the course website, and in general may be accessible by the public. The original reports will not be released but will be accessible only to the course instructors and administrative staff.

Participation is completely voluntary and students may refuse to participate or may withdraw their permission at any time. Reports will only be used with the signed consent of all team members. Participating will have no influence on the grading of your work and there is no penalty for not taking part.

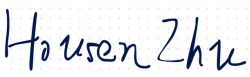
If your group agrees to take part, please have all members sign the bottom of this form.

Consent Statement

We verify that we have read the above letter and are giving permission for the ECE496 course coordinator to use our reports as outlined above.

Team #: 2024105 Project Title: Building an Intelligent Job Scheduler for Optimized Request Assignment

Supervisor(s): Ali Bereyhi Administrator: Sarah Oliveira

Name	**Yifan Wang**	**Signature**		**Date:**	**3/21/2025**
Name	**Housen Zhu**	**Signature**		**Date:**	**3/21/2025**
Name	**Ruoyu Li**	**Signature**		**Date:**	**3/21/2025**
Name	**Yuan Sui**	**Signature**		**Date:**	**3/21/2025**

Group Highlights（author: Ruoyu Li）

Our team has successfully developed both the front-end and back-end components of the project. On the front end, we designed a user-friendly main website for product introduction and key section navigation, a unified login page supporting both patient and doctor access, a patient chatbot for information acquisition, and a doctor interface for data management. Despite having no prior experience with frontend development, our team successfully built a visually appealing and dynamic website using only HTML and CSS, without relying on any advanced frontend frameworks such as React or Vue. By focusing on simplicity and leveraging core web technologies, we were able to design a user-friendly interface that includes several interactive features. For instance, we implemented CSS-based visual effects such as fade-in transitions and scroll animations to enhance the user experience and give the site a more modern and responsive feel. This approach demonstrates that with creativity and a good understanding of basic web design principles, it is still possible to achieve a clean, professional frontend without the overhead of complex frameworks. By keeping the webpage loading time under 1 second and buttons response time under 0.1 second, we ensure a fluent user experience when interacting with our frontend. On the back end, we implemented the DQN model and trained it using randomly generated data sampled from a real-world distribution. Our model has demonstrated a significant reduction in weighted delay compared to the best-performing rule-based algorithm, therefore effectively improving the scheduler output. After the implementation, we also did systematic testing and evaluation for both the front-end and back-end and confirmed the functionality and usability of our project.

Individual Contributions

Contributions from Ruoyu Li (author: Ruoyu Li)

My major role in this project was developing the front end: designing and implementing web pages and the doctor's interface, connecting them with the backend, and running debugging tests to assure the applications are working.

Initially, we planned to use Gradio as our front-end framework. It was very straightforward and provided native support for embedding machine learning models into applications. However, very quickly, we hit deadlocks over the issue of styling with CSS, as the framework had prebuilt styles that kept overriding. After a while, in the winter break, I decided to scratch and build a new front end for my application using HTML and CSS.

In the winter break, I built the website: a header bar, explore, advantages, about us, and contact us section. I also built a login page redirecting to the patient chatbot for patients and the doctor interface for doctors. I built an interface for the doctor with a sidebar to control and an information display area. The sidebar controls several input forms through which a doctor can update information regarding him or his patients. The information container is supposed to show dynamic information regarding updated status and treatment plans.

I also worked on integrating the front end with the backend algorithm. I developed app.py to connect the front-end templates to the backend algorithm. App.py pulls data from the database at half-second intervals and pushes the data into the front-end templates for display. It sends user input from template forms to the backend database and waits for the algorithm to execute based on the data it just received. This integration ensures seamless communication between components and enables the project to function as a cohesive system.

Contributions from Yifan Wang (author: Yifan Wang)

In this project, my primary responsibility for this project was acting as a bridge for communication between the frontend and backend teams, thus promoting collaborative work and optimizing operational efficiency. To accomplish this purpose, I worked systematically on enhancing the Gantt chart within the Implementation Plan through dividing duties into smaller portions. Such a systematic approach made it easier to monitor progress and stay on schedule with project objectives. For this project, I also scheduled and facilitated weekly meetings with a view to assessing milestones, solving problems, and synchronizing team operations. One notable achievement from this undertaking was acquiring assurance that all functional deployments were finalized by the back-end team, with the front-end team concluding model training before March. Such a well-thought-out plan highly contributed towards productivity and optimized our developmental process.

Apart from managing the activity, my active involvement with the project involved two main areas. First, I assisted the frontend team through the use of artificial intelligence tools, which helped beautify the website with a variety of artistic features. The beautification process was achieved through implementing Stable Diffusion, a generative AI tool praised for its capability for producing images with satisfactory quality as well as for avoiding copyright infringement. The integration of these tools not only beautified the website but also created a personalized experience for the users.

Secondly, I designed the functions of the chatbot to engage with patients. From this perspective, the chatbot became one of the patient-facing components of the project as it directly provided responses to health inquiries. To favor accuracy and reliability, the knowledge base was sourced from recognized materials, including the Oxford Medical Textbooks, reputed for having voluminous and up-to-date medical information. While not the main focus of this work, substantial effort went into making the responses of the chatbot clear and accurate. Some medical queries were, however, constructed in such a way that they called for recommendations for immediate consultation with the appropriate medical profession, thus ensuring the system's reliable and safe operation.

Contributions from Housen Zhu (author: Housen Zhu)

My primary responsibility in Fall term was to ensure the correctness and reliability of the simulated environment in which the agent would be trained. I tested various scenarios and edge cases to verify that the environment behaved as expected under different conditions.

In parallel with the environment validation, I contributed to the building and training process of the DQN model. I created the initial training pipeline, defined the state and action spaces, and tested the reward functions to guide the learning of the agent. I was actively engaged with the hyperparameter tuning, checking that the training was proceeding as expected, and observing how the agent was learning. My main goal was to build the model in such a manner that it would learn optimally and get an optimal policy faster, so that the agent would not waste time or resources during training.

In the winter term, I focused on fine-tuning the model so that it performed as optimally as could be. I adjusted parameters like the learning rate, epsilon decaying rate, and worked together with training techniques like memory replay to make the learning process stable. I also tested how rewards were being shaped and made changes to allow the agent to learn more quickly and effectively. It was a lot of trial and error, but once I tested and tweaked it thoroughly, I was able to get the model's performance optimized.

Also, I was responsible for evaluating the performance of the model. I tested the agent in various situations and established how effectively it was able to make the right choices and adapt to finding solutions to new problems. I created the evaluation strategy on running the agent on four different incoming patient densities. Then compare its performance with other rule-based scheduling algorithms on the average weighted waiting time of ED patients. The successful evaluation process ensures the model aligns with the project goals and is able to respond under different scenarios.

Contributions from Yuan Sui (author: Yuan Sui)

At the beginning of the fall term, I developed the training dataset by generating realistic patient information, including acuity levels, arrival times, treatment plans, and treatment durations. Using statistical models that reflect real-world hospital inflow, I ensured the data accurately represented emergency department dynamics, providing a solid foundation for training the reinforcement learning model.

In addition, I was also involved in the initial design of the backend algorithm structure. This included designing and building the hospital simulation environment, integrating key factors that influence patient flow and resource allocation. By refining the environment's parameters, I helped create a more realistic setting for testing and improving the model's decision-making process.

As the project progressed, I was responsible for running the model and fine-tuning hyperparameters to optimize its performance. By systematically adjusting key parameters, I ensured the reinforcement learning algorithm effectively adapted to different patient scheduling scenarios. This process was critical in enhancing the model's ability to make efficient and accurate decisions in a dynamic hospital setting.

Beyond implementation, I deepened my understanding of reinforcement learning algorithms and relevant research, applying these insights to refine the model's design. My contributions helped bridge theoretical concepts with practical application, ultimately enhancing the algorithm's effectiveness in optimizing patient scheduling and reducing wait times.

Acknowledgements (author: Housen Zhu)

We would be more than happy to offer our sincerest appreciation to our project supervisor, professor Ali Bereyhi, for his invaluable contributions, constant guidance, and expert counsel. His vast knowledge of reinforcement learning and Markov decision processes were critical in shaping the course of our work and enabling us to steer through the problems that we faced. The useful comments and advice provided by professor Bereyhi were essential in the improvement of work quality and in motivating us to do better.

We are also indebted to the Department of Electrical and Computer Engineering at University of Toronto for providing the resources and platforms for the success of this project. The collaborative potential, learning, and growth within the ECE department have contributed significantly towards shaping our academic life.

Especially thanks to our peers and fellow students who shared ideas, provided feedback, and participated in discussions that improved our project. They forced us to look at the issue from fresh perspectives and catalyzed creative solutions to complex issues in emergency department scheduling.

We would also like to thank the participants who enabled the testing and evaluation process of our system, including simulated users and volunteers. Their feedback was greatly helpful in improving the usability, functionality, and overall user experience of our website interface and chatbot.

1. Executive Summary (author: Yuan Sui)

Our team has successfully developed an intelligent emergency department (ED) patient scheduling system that integrates a deep reinforcement learning (DRL)-based scheduler with an intuitive front-end interface. The project aims to optimize patient triage and resource allocation to reduce waiting times while maintaining fairness and efficiency.

On the front end, we designed a responsive web-based interface that includes a main website for system navigation, a unified login page for both doctors and patients, an interactive chatbot for patient inquiries, and a doctor's dashboard for real-time data management. The user experience was optimized by maintaining webpage load times under 1 second and button response times below 0.1 seconds, ensuring seamless interaction.

On the back end, we developed and implemented a Deep Q-Network (DQN)-based scheduling algorithm. The model was trained using synthetic data sampled from a real-world patient arrival distribution, allowing it to dynamically adjust to varying levels of patient acuity and hospital resource constraints. Experimental results demonstrated that our DQN-based scheduler significantly reduces weighted patient wait times compared to conventional rule-based scheduling algorithms, striking an effective balance between prioritizing critical patients and maintaining overall system efficiency.

To ensure robustness, we conducted extensive systematic testing and validation of both the front-end and back-end components. The web interface was rigorously evaluated for functionality, usability, and performance, while the DQN model was tested against real-world-inspired patient flow scenarios. Comparisons with traditional scheduling methods confirmed the superior adaptability and efficiency of our approach.

Our project highlights the potential of AI-driven solutions in hospital operations management and lays the groundwork for future improvements, including real-time data integration from hospital systems, advanced multi-modal learning approaches, and eventual real-world deployment in hospital settings.

2. Table of Contents (author: Ruoyu Li)

Group Final Report Attribution Table **1**
Voluntary Document Release Consent Form **3**
Group Highlights（author: Ruoyu Li） **6**
Individual Contributions 7
Contributions from Ruoyu Li (author: Ruoyu Li) 7
Contributions from Yifan Wang (author: Yifan Wang) 8
Contributions from Housen Zhu (author: Housen Zhu) 9
Contributions from Yuan Sui (author: Yuan Sui) 10
Acknowledgements (author: Housen Zhu) **11**
1. Executive Summary (author: Yuan Sui) **12**
2. Table of Contents (author: Ruoyu Li) **13**
3. Introduction (author: Yifan Wang) **15**
4. Final Design (author: Yuan Sui, Housen Zhu, Ruoyu Li, Yifan Wang) **17**
4.1 System-Level Overview (author: Yifan Wang, Yuan Sui) 17
4.1.1 Graphical User Interface Overview (author: Yifan Wang) 17
4.1.2 Backend Overview (author: Yuan Sui) 17
4.2 System Block Diagram (author: Yifan Wang) 19
4.3 Module-Level Overview (author: Yuan Sui, Housen Zhu, Ruoyu Li, Yifan Wang) 20
4.3.1 Emergency Department Simulator Module (author: Yuan Sui) 20
4.3.2 DQN Model Design (author: Housen Zhu) 20
4.3.3 Deep Reinforcement Learning Training Technique (author: Housen Zhu) 21
4.3.3.1 Epsilon-Greedy Policy (author: Housen Zhu) 21
4.3.3.2 Memory Replay (author: Housen Zhu) 22
4.3.4 Graphical User Interface Design (author: Ruoyu Li) 22
4.3.4.1 The Main Website (author: Ruoyu Li) 23
4.3.4.2 Login Page and Doctor Interface (author: Ruoyu Li) 25
4.3.5 Chatbot Design (author: Yifan Wang) 26
4.4 Final Design Assessment (author: Housen Zhu, Yuan Sui) 27
4.4.1 Effectiveness and Performance Analysis (author: Housen Zhu) 27
4.4.2 Limitations (author: Yuan Sui) 28
5. Testing and Verification (author: Yuan Sui, Housen Zhu) **29**
5.1 Training and Testing Dataset (author: Yuan Sui) 29
5.2 DRL Model Test and Verification (author: Housen Zhu) 30
5.2.1 Algorithm Performance Evaluation (author: Housen Zhu) 30
5.2.2 DRL model Testing Table (author: Housen Zhu) 32
5.3 Graphical User Interface Test and Verification (author: Ruoyu Li) 32

5.3.1 Website Testing (author: Ruoyu Li) 32
5.3.2 Graphical User Interface Testing Table (author: Ruoyu Li) 33
5.4 Chatbot Test and Verification (author: Yifan Wang) 34
5.4.1 Chatbot Functional Testing (author: Yifan Wang) 34
5.4.2 Chatbot Performance Testing (author: Yifan Wang) 35
5.4.3 Chatbot Testing Table (author: Yifan Wang) 36
6. Summary and Conclusions (author: Yuan Sui) 37
7. Future Work (author: Yifan Wang) 38
8. References (author: Ruoyu Li) 39
9. Appendices (author: Housen Zhu) 42
A: Gantt chart History Appendix 42
B: Financial plan (updated from Implementation Plan) 43
C: Complete training algorithm [14] 44
D: Rule-based algorithm used to compare with our deep reinforcement learning approach [14] 45
E: Emergency Department Simulator 46

3. Introduction (author: Yifan Wang)

The Emergency departments (ED) is a complicated system, including the constraints of medical resources and the clinical condition of patients, which are interconnected. Furthermore, the occurrences in an ED are hard to predict due to the nature of unscheduled patient arrivals. Thus, it is hard to pre-assign adequate medical resources. With such complexity and unforeseen circumstances, errors on the part of decision-makers are highly probable, which can adversely affect the treatment process of the patient. In some extreme cases, one wrong decision can double the waiting time of patients, causing overcrowding in the ED [1]. Correspondingly, if patients' waiting time in the ED is extended, their health condition might deteriorate, which may lead to severe adverse consequences.

The first thing that needs to be clarified is the patient's condition when addressing this issue. According to the Canadian Triage and Acuity Scale (CTAS) [2], the acuity of a patient is classified into five levels of severity, from 1 to 5, based on the urgency required to alleviate pain and manage life-threatening conditions. Notably, acuity level 1 is for a patient who needs immediate medical attention. For acuity levels ranging from 2 to 5, the urgency associated with patient treatment and the severity of the disease progressively diminish in importance [Figure 1]. Nevertheless, treating such patients with high acuity levels is not necessarily the most rewarding strategy for health facilities, as the others with low acuity levels must be accorded appropriate care. Though the longer waiting times might not worsen the clinical status of patients, delays in treatment can cause dissatisfaction among patients with low levels of acuity, to the detriment of the reputation of the hospital. As such, waiting time management in the emergency department requires the assessment of all levels of acuity.

Level	Urgency	Estimated Time to be seen	Place
Level 1	Resuscitation	Immediately	Emergency
Level 2	Emergency	Within 15 min	Emergency
Level 3	urgency	Within 30 min	Emergency
Level 4	Less urgency	Within 60 min	Local health center
Level 5	Non urgency	Within 120 min	Local health center

Figure 1: Canadian Triage and Acuity Scale System [3]

For decades, there have been concerted efforts to tackle the crowding issue because of complexities in EDs. Governments, for example, have set targets/indicators related to performance [4]. Other researchers have suggested different types of mathematical models using programming. They have been used in the fields of operating room scheduling [5], nursing scheduling [6], and emergency department discrete event simulation [7]. However, waiting time issues continue to affect EDs because there are few appropriate strategies to respond to the varying severity of patients.

Recently, advancement in deep reinforcement learning has enabled new algorithms to leverage trends in past data to forecast patient demand and schedule accordingly to significantly reduce wait time. Currently, the state of art scheduler for teleconsultation utilizes Deep Q Network (DQN) and achieved 9% – 41% reduction in average waiting time [8]. In this paper, we will utilize deep reinforcement learning (DRL) to enable patient scheduling in an emergency department with limited medical resources in a dynamic setting. The following sections will outline the technical framework, testing and verification results, and a critical assessment of the final design. Additionally, recommendations for future improvements and potential real-world applications will be discussed.

4. Final Design (author: Yuan Sui, Housen Zhu, Ruoyu Li, Yifan Wang)

4.1 System-Level Overview (author: Yifan Wang, Yuan Sui)

4.1.1 Graphical User Interface Overview (author: Yifan Wang)

The system's front end is designed as the primary user interface, serving as the gateway to an intelligent emergency department job scheduling system that optimizes request allocations. It is constructed with simplicity and functionality in mind as a fundamental principle, ensuring the interface is accessible, responsive, and highly optimized for the special requirements of different user populations, including patients and medical staff. The front-end framework is based on four core elements: the home page (See section 4.3.4.1), the login page (See section 4.3.4.2), the scheduler interface (See section 4.3.4.2), and the chatbot module (See section 4.3.5), with each component meticulously crafted to foster a seamless user experience.

In parallel, the system provides a dedicated chatbot interface for patients, called MediGuide. This component is integral to the user experience, offering real-time updates and interactive responses that address common health related problems. It employs natural language processing techniques to interpret and respond to queries in a conversational manner, ensuring that patients receive accurate and timely information.

4.1.2 Backend Overview (author: Yuan Sui)

The implementation of the backend algorithm unfolds over three distinct yet interrelated phases: design, training, and integration. In the design phase, the development team employs Python to create a backend system composed of two fundamental modules: an Emergency Department (ED) simulator (See section 4.3.1) and a Deep Reinforcement Learning (DRL) algorithm module (See section 4.3.2). Specifically, the ED simulator keeps track of patients waiting for each medical resource and the state of occupancy for each resource. The DRL algorithm selects the next patient to be attended by the medical resource and updates the ED simulator of the decision. The ED simulator will

then manage the new state of occupancy for each medical resource according to the DRL algorithm decision.

In the second training phase, the team will generate realistic patient data (e.g. arrival time and treatment pattern) that is statistically similar to what is observed in hospital emergency departments. The generated data will be used to train the algorithm to learn how to best make decisions to minimize patient wait time. Each time the algorithm makes a patient selection, the decision and new state of medical resources is stored in the memory module. After a number of decisions are made, the algorithm learns from past memory and updates DRL algorithm weights. With hours of generated data training, the Q-network will be optimized to make the selection that minimizes patient waiting time in the hospital.

In the final integration phase, the team will connect the backend scheduling algorithm to the frontend GUI for ease of use in the hospital setting. As a patient arrives, their information will be logged in to the system and the algorithm will automate the scheduling of medical resources. The training of algorithms continues during deployment and will become even more optimized as the network learns and adapts to the specific hospital environment.

4.2 System Block Diagram (author: Yifan Wang)

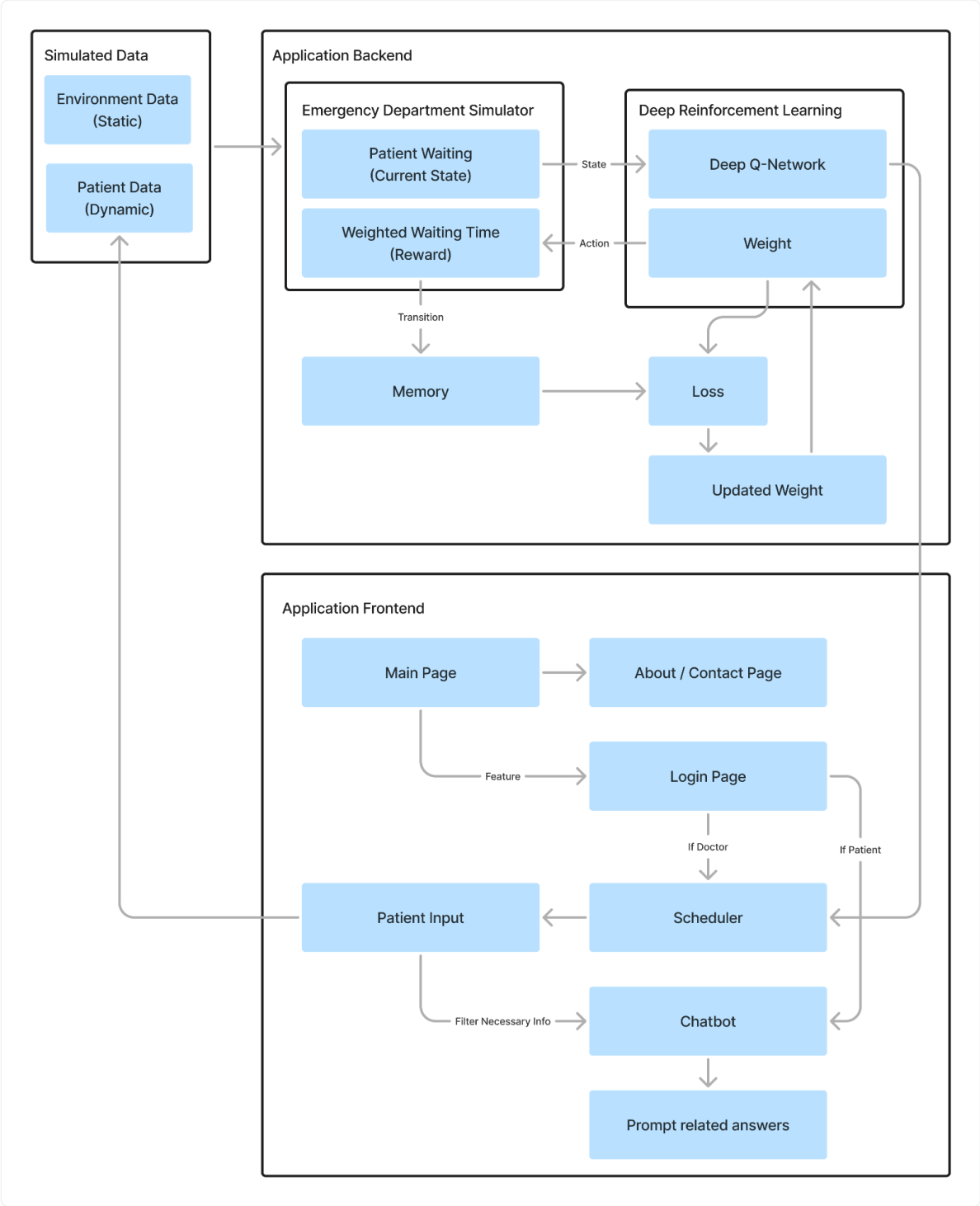


Figure 2: System block diagram

4.3 Module-Level Overview (author: Yuan Sui, Housen Zhu, Ruoyu Li, Yifan Wang)

4.3.1 Emergency Department Simulator Module (author: Yuan Sui)

The Emergency Department (ED) Simulator serves as a virtual representation of an emergency room, dynamically tracking patient flow and resource utilization (See Appendix E). It constructs data structures that monitor the occupancy status of medical resources—doctors, nurses, X-ray rooms, and CT rooms—updating in real time as patients arrive, receive treatment, and depart. At any given moment, the simulator provides a snapshot of patient wait times for each resource, enabling a data-driven approach to optimizing hospital operations.

In the simulation setup, each medical resource has a predefined number of available slots. For example, the system may allocate three doctors, a fixed number of nurses, and designated X-ray and CT rooms. Each resource is linked to specific treatment types; for instance, patients requiring registration will be assigned to nurses. These resource constraints and patient assignments create a dynamic environment where bottlenecks and inefficiencies can be analyzed and addressed.

The module's configurations are highly flexible, allowing adjustments to align with real-world hospital conditions. By modifying the number of available resources, patient inflow rates, and treatment durations, the simulator can model different emergency room scenarios. This adaptability ensures that the module remains relevant for various healthcare settings, providing a robust foundation for testing and improving patient flow management strategies.

4.3.2 DQN Model Design (author: Housen Zhu)

As shown in Figure 3, the Deep Q-Network model of this project uses a fully connected neural network (FCNN). These neural network layers are employed to detect encoding current status of the emergency department (ED) such as patient acuity levels, waiting time, and available resources.

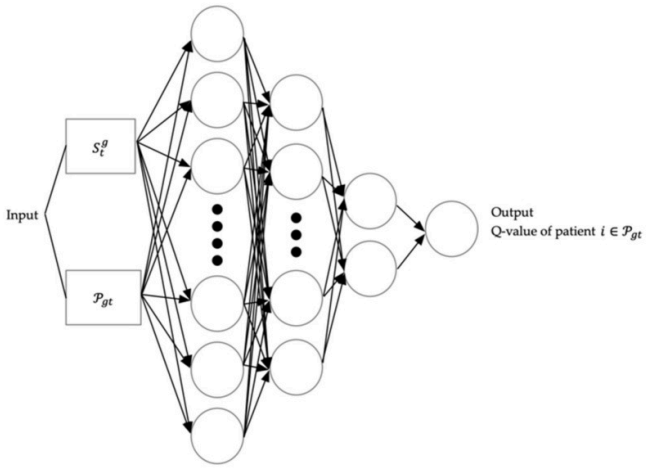


Figure 3: architecture of DQN model [14]

4.3.3 Deep Reinforcement Learning Training Technique (author: Housen Zhu)

Due to the space limit, we only talk about two training techniques below. The complete training algorithm shown in Appendix C.

4.3.3.1 Epsilon-Greedy Policy (author: Housen Zhu)

To maintain equilibrium between exploration and exploitation during training time, the agent uses an epsilon-greedy policy for selecting actions in Figure 4. With probability ε (epsilon) at each decision step, the agent selects a random action for encouragement of exploration of the action space and to prevent hasty convergence to a suboptimal policy. Otherwise, with probability 1 - ε, the agent exploits what it knows by choosing the action with highest Q-value predicted by the model. This balance ensures the agent keeps on learning potentially superior actions while its policy is being updated from accumulated experience. In training, epsilon is gradually reduced from an initial high value towards a lower level, allowing the agent to move from exploration-oriented behavior to exploitation as it becomes sure of the policy it has learned.

Input: S_t^g, P_{gt}

Output: A patient with maximum Q-value

rn ← random ( ):

  if rn $\leq \epsilon$ then

    action ← random (P_{gt})

  else

    action ← maxQvalue (S_t^g, P_{gt})

  end if

Figure 4: action selection algorithm using Epsilon-Greedy Policy [14]

4.3.3.2 Memory Replay (author: Housen Zhu)

For enhancing the stability and efficiency of learning, the DQN model utilizes a memory replay system. During training, the agent consumes past experiences—each consisting of the state, action taken, reward received, and next state—and stores them in a replay buffer. Instead of using the newest experience for updates to the model only, as shown in Figure 5, the agent randomly samples mini-batches of experiences from this buffer and uses them for performing training. This random sampling breaks the correlation of sequence between experiences, reducing the likelihood of biased learning and improving the stability of gradient updates.

if $|M| \geq \frac{L}{2}$ and $t\%60 = 0$ then

  Sampling random minibatch (s_h, a_h, r_h, s_{h+1}) from M

Figure 5: minibatch sampling using Memory Replay [14]

4.3.4 Graphical User Interface Design (author: Ruoyu Li)

The graphical user interface (GUI) plays a critical role in our capstone project, as it serves as the primary point of interaction between users and our system.

4.3.4.1 The Main Website (author: Ruoyu Li)

The top-level structure of the main website was divided into five classes, all aimed at progressively improving usability and access to information. Above the fold, in a sticky header, four buttons allow access to key information areas from anywhere on the site simply and intuitively.



Figure 6: Main Website

Right under the header sits the exploration section, which contains a captivating background image and tagline. To this portion of the webpage, we added an appropriate fade-in effect to the tagline. The button to "Learn More" about the scheduler redirects us to the project's GitHub page for more information.

Scrolling down, users will see a very brief section touting this website's benefits. Simple images accompany a brief explanation of each benefit in a clean and minimalistic manner, not trying to overload the user with an overabundance of detail.

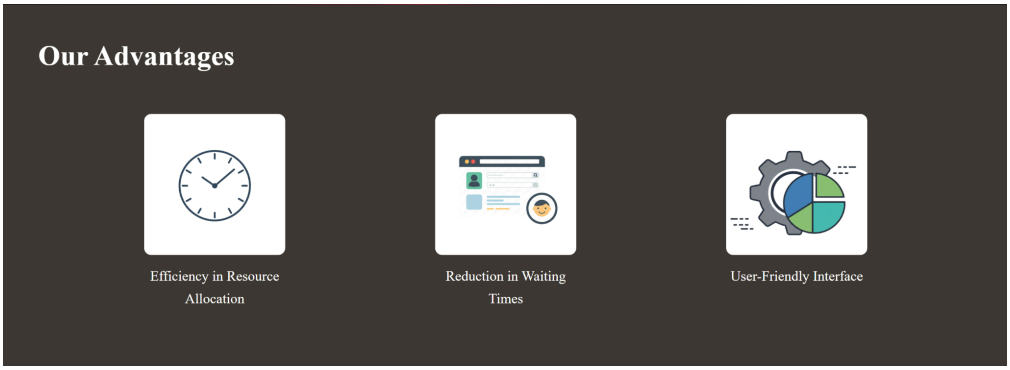


Figure 7: Advantage Section

This is followed by a larger introductory section divided into four parts. This section will give the user a step-by-step guide on how the doctor interface works and what it is. To keep the user from getting lost and to develop this illusion of "the continuation of the page, the scroll-triggered effect is used. As the user scrolls, the new section slides up from below, overlapping the last slide to keep the user's attention in the center of the screen.

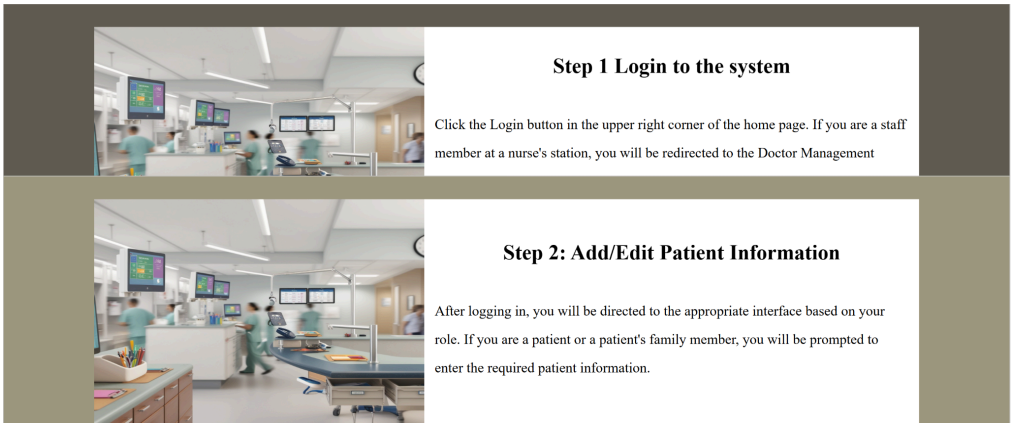


Figure 8: Introduction Section

A contact section is included. Here, a user can find details to contact the team. At the bottom of each page is a small footer line signifying copyright, a professional finish to the website.

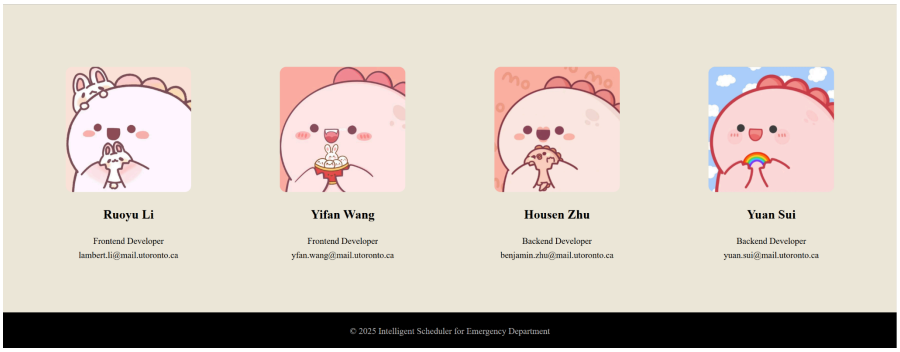


Figure 9: Contact Section

4.3.4.2 Login Page and Doctor Interface (author: Ruoyu Li)

In addition to the main website, we also designed the login page and doctor interface for operating the scheduler. The login page allows doctors to sign in and access the interface. The doctor interface consists of a side control bar and a status display section. The control bar is where the doctors can perform key operations such as adding new patients. The status display section shows real-time information on doctors and patients. Users can toggle between viewing doctor and patient status through a top control bar or search for a specific individual.

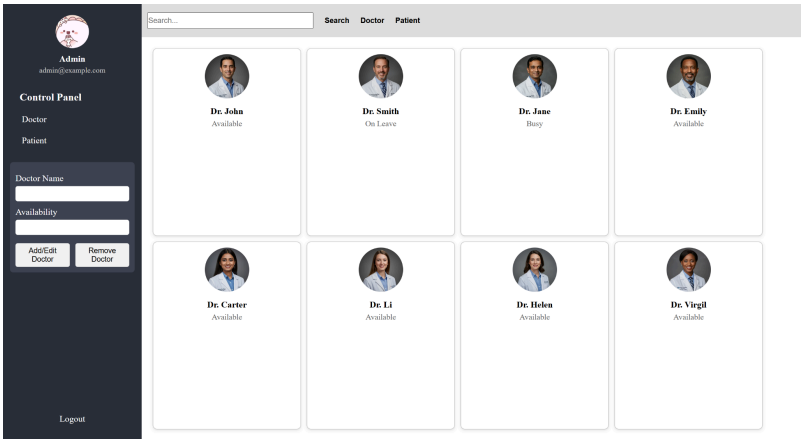


Figure 10: Doctor Interface

4.3.5 Chatbot Design (author: Yifan Wang)

The MediGuide chatbot module is a crucial component of our intelligent job scheduling system, specifically designed to enhance patient interaction by providing real-time, evidence-based responses to health-related inquiries. To achieve both accuracy and efficiency, the chatbot is built by integrating a structured medical knowledge base directly on top of an existing large language model. This approach leverages the advanced natural language understanding capabilities of large-scale models while ensuring that the responses generated align with verified medical information.

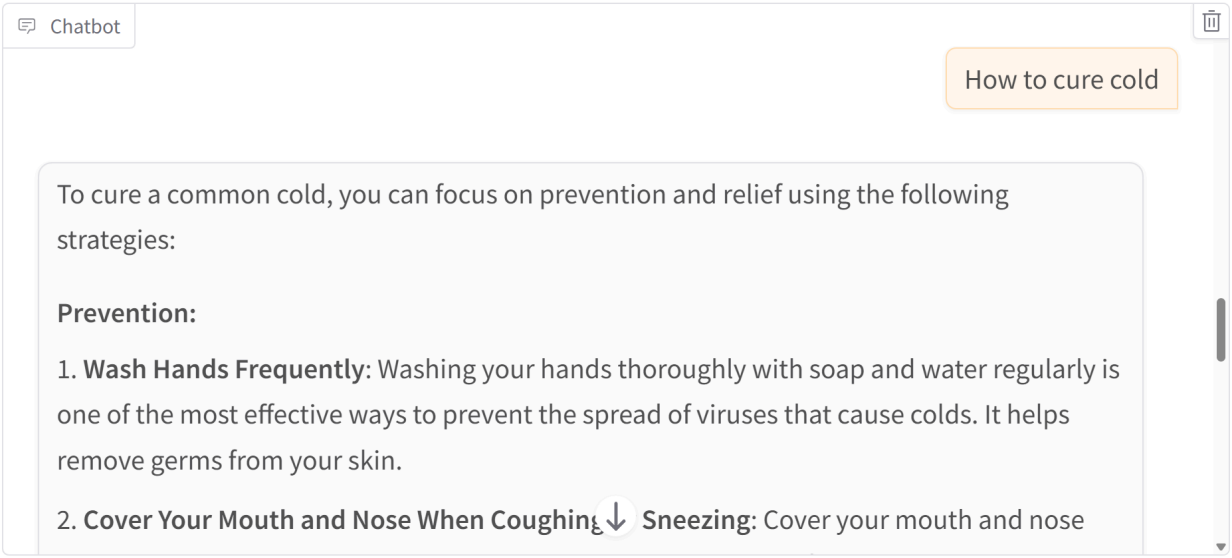


Figure 11: Screenshot of MediGuide chatbot

To maintain a high standard of medical accuracy, the chatbot's knowledge repository is constructed using reputable sources such as Oxford Medical Textbooks. The architecture of the chatbot knowledge base is structured into multiple layers to optimize efficiency, accuracy, and scalability. At its foundation, a curated repository of trusted medical literature ensures that responses remain aligned with professional healthcare guidelines. The knowledge base is categorized into medical topics, allowing for fast retrieval of relevant information. Additionally, when a query involves specialized or high-risk medical concerns, the chatbot intelligently redirects the patient to consult a healthcare professional. This hierarchical decision-making process ensures that critical

healthcare decisions remain within the expertise of licensed practitioners while still allowing the chatbot to efficiently address general health concerns. Overall, the chatbot serves as an intelligent, reliable, and user-friendly interface that enhances patient engagement by leveraging both pre-trained linguistic models and domain-specific knowledge.

4.4 Final Design Assessment (author: Housen Zhu, Yuan Sui)

4.4.1 Effectiveness and Performance Analysis (author: Housen Zhu)

The system proposed is a comprehensive and pragmatic approach to solving patient scheduling issues in the emergency department (ED) by integrating a Deep Reinforcement Learning model (DQN), an easy-to-use website, and a smart chatbot. The effectiveness of the project is that it can enhance decision-making, user experience, and operating efficiency in a high-stakes environment in healthcare.

The key purpose of the system, the intelligent scheduler, is minimizing the weighted wait time of patients. By considering patient severity, arrival times, and resource availability, the model learns to deal with the stochastic nature of ED operations and optimally prioritize the patients. Data-driven decision-making improves patient flow management, reduces bottlenecks, and facilitates timely treatment of critical cases—functioning directly to enhance healthcare outcomes. By implementing the DQN model, the weighted delay is reduced by half compared with the most efficient rule-based algorithm. This huge improvement in performance further confirmed the effectiveness of our project.

The website interface strongly encourages the usability and accessibility of the system. Through the enabling of both doctors and patients to login and interact with the scheduler, the system enables a transparent, real-time update. Doctors have access to see real-time schedules and lists of waiting patients. At the same time, patients are also advantaged by being able to see where they stand in the queue, get estimated waiting

times, and be kept informed during their visit. For the performance of the website interface, we always ensure the webpage loading time below 1 second and button response time below 0.1 second, this also confirms the usability of the website.

Additionally, the smart chatbot further improves the effectiveness of the system through its 24/7 assistance. The chatbot also improves patient engagement through providing real-time responses to medical inquiries without staff intervention. The administrative burden on healthcare personnel is reduced, and the patient experience is improved as a whole.

4.4.2 Limitations (author: Yuan Sui)

The implementation of reinforcement learning (RL) in hospital scheduling, particularly for emergency departments (EDs), encounters a couple limitations compared to the weighted rule-based system assessed in this study. The rule-based approach based on predefined criteria, offers predictability and stability suited to consistent demands. RL, while capable of dynamically optimizing schedules through iterative learning, requires real-time feedback on patient conditions and resources. Without continuous updates on patient conditions or resource availability, RL struggles to adapt scheduling decisions to the rapidly evolving demands of an ED, thereby limiting its practical utility relative to the established rule-based framework.

Further constraints on the deployment of RL in hospital scheduling arise from data deficiencies and simplifying assumptions inherent in the study's ED model. Certain detailed information on medical resources such as treatment interruptions, staff skill differentiation, and bed allocation for low-acuity patients are not available in our ED model. These gaps, compounded by the omission of interconnected departments like operating rooms, restrict the fidelity of the simulation environment necessary for RL to optimize resource allocation or respond to demand fluctuations.

5. Testing and Verification (author: Yuan Sui, Housen Zhu)

5.1 Training and Testing Dataset (author: Yuan Sui)

To ensure the accuracy and robustness of our reinforcement learning algorithm, we constructed a comprehensive dataset that simulates patient flow and emergency department (ED) resource availability. The dataset consists of two key components: simulated environment data and simulated patient data, both of which serve as the foundation for training and validating the model.

The simulated environment data encapsulates the static parameters of a hospital emergency room, including the number of doctors, nurses, CT technicians, and other medical resources. These values remain constant throughout the simulation, allowing us to test the model's ability to optimize resource allocation under fixed constraints. By maintaining a stable environment, we can isolate the impact of patient dynamics on the system's performance.

The simulated patient data is generated using statistical models that reflect real-world patient inflow patterns. During the training phase, we employ data-driven techniques to create diverse patient profiles, including acuity levels, arrival times, treatment types, and expected treatment durations. This synthetic dataset enables the algorithm to learn complex patterns in patient distribution and hospital congestion, beyond what human administrators might intuitively detect. In the deployment phase, however, patient data will be dynamically entered by hospital staff in real time, allowing the system to adapt to actual hospital conditions and continuously refine its scheduling decisions.

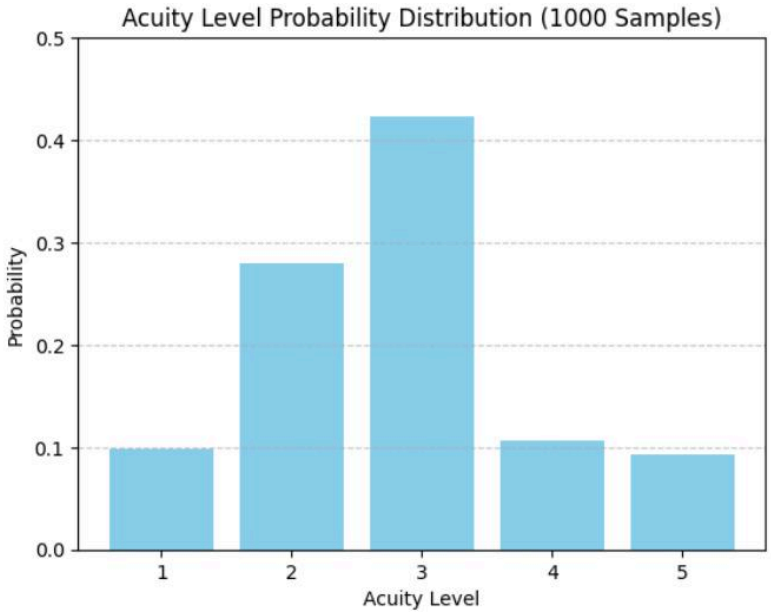


Figure 12: Example of patient acuity level distribution in the training dataset

5.2 DRL Model Test and Verification (author: Housen Zhu)

5.2.1 Algorithm Performance Evaluation (author: Housen Zhu)

By considering the variability of patient flow density and acuity score, our test strategy focused both on the quantitative performance of the model as well as the model's qualitative response across varying decision-making contexts. We compared the DQN agent with a variety of baseline policies. The baselines employed were simple heuristics like First-Come-First-Served (FCFS), Shortest-Remaining-Process-Time (SRPT), and other comprehensive rule-based triage policy (explained in Appendix D). The comparison result showed in Figure 12 provided us with a sharp insight into the DQN's advantage in minimizing the weighted waiting time of ED patients.

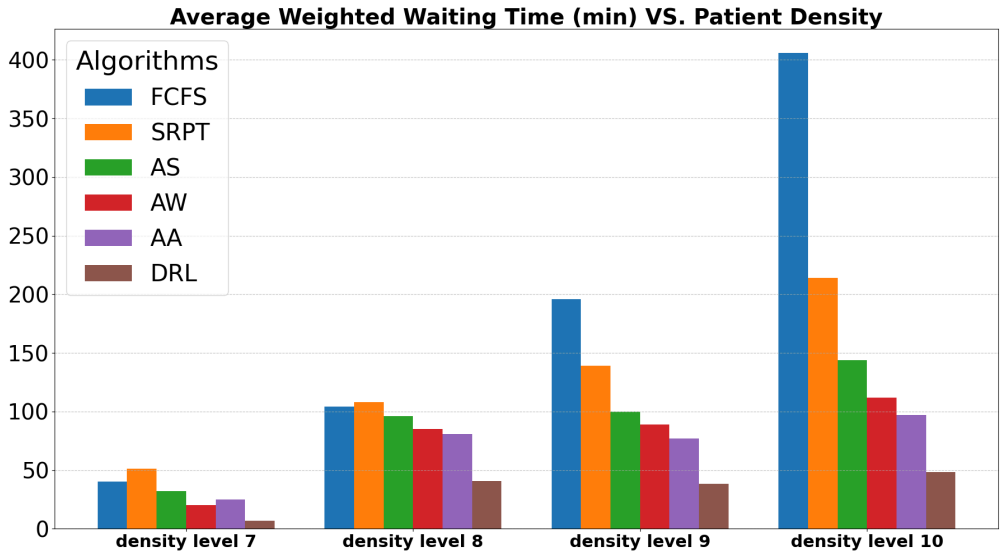


Figure 13: the comparison of performance between different algorithms and incoming patient density levels

Table 1 presents the experimental results on percentage of penalty patients in acuity levels 1 and 2 for all density scenarios. Less than 5% of the patients exceed the threshold (60 mins) when using DRL [9].

Table 1: number of patients (%) exceed penalty threshold

Density Level	FCFS	SRPT	AS	AW	AA	DRL
7	43.9	29.3	16.1	5.9	12.2	2.9
8	100	45.4	23.2	9.0	24.9	4.4
9	100	43.0	24.9	37.5	26.1	4.6
10	100	36.4	35.4	66.8	18.9	4.6

5.2.2 DRL model Testing Table (author: Housen Zhu)

ID	Requirement	Method	Result and Proof
3.1	Lower waiting time than rule-based algorithms	Running comparison testing on different patient densities (Figure 13)	Pass. Achieve 40% improvement than all schedules
3.2	Lower number of penalty patients in acuity 1 and 2	Running comparison testing on different patient densities (Figure 13)	Pass. Only 5% penalty patients in acuity 1 and 2

5.3 Graphical User Interface Test and Verification (author: Ruoyu Li)

5.3.1 Website Testing (author: Ruoyu Li)

The website testing strategy for our project adopts a clear stepwise methodology for validating the functionality, usability, and performance of the graphical user interface. The testing process includes manual verification of significant interactive elements, functional testing of navigation and authentication, and performance tests to measure response times. We also give special attention to button and animation functioning, as well as components that depend on data, for smooth user interaction. The login system and doctor interface were tested in terms of handling proper authentication, real-time updates, and database synchronization. The following table provides detailed individual test cases, test methods, and their results.

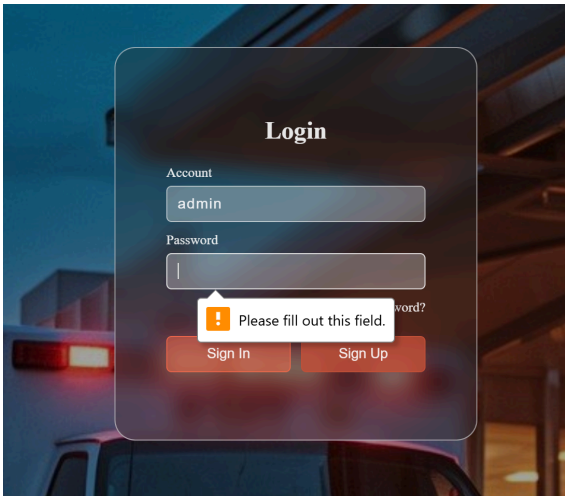


Figure 14: Error Message in Login Page

5.3.2 Graphical User Interface Testing Table (author: Ruoyu Li)

Table 3: Graphical User Interface Testing Table

ID	Requirement	Method	Result and Proof
4.1	Header bar stay fixed when scrolling	Manually Scroll down and inspect the header bar	PASS. Fixed.
4.2	Navigation buttons link to the correct sections	Click each navigation button and verify correct section scroll	PASS. Correct sections reached.
4.3	Explore section slogan fades in correctly	Load the page and observe if the fade-in effect appears	PASS. Fade-in effect appeared.
4.4	Learn More button redirects to Github Page	Click Learn More button and confirm redirection to Github	PASS. Github redirected.
4.5	Login form accept valid credentials	Enter correct credentials and check redirection	PASS. Credential accepted and

	and redirects		redirected.
4.6	Invalid login shows error message	Enter incorrect credentials and verify error message	PASS. Error message showed up. See Figure 14.
4.7	Doctor interface displays real-time patient/doctor status	Check backend patient database and interface status display at the same time and confirm it	PASS. Patient info display is aligned with database.
4.8	Doctor interface pushes user input correctly	Add both patient and doctor data using side panel and confirm input added into backend database	PASS. Input data inserted into the database.
4.9	Button response time less than 0.1s[10]	Add time function in frontend to measure response time	PASS. Response time around 0.003s.
4.10	The website takes user less than 3 clicks to access doctor interface	Count the clicks needed to access doctor interface	PASS. Two clicks needed.

5.4 Chatbot Test and Verification (author: Yifan Wang)

The verification and testing of the chatbot, MediGuide, involves functional testing, performance testing, and a chatbot testing table.

5.4.1 Chatbot Functional Testing (author: Yifan Wang)

Functional testing of the chatbot is conducted to verify that each component and feature operates according to the specified requirements. This phase systematically examines the chatbot's ability to accurately process user inputs, generate contextually appropriate

responses, and interact with the backend scheduling system. In a formal testing environment, test cases are designed to cover a range of scenarios, including common health inquiries, edge cases, and error conditions. The screenshot demonstrates that the model is designed to categorize patient queries based on complexity [Figure 15]. General health enquiries, such as advice on managing mild symptoms, can receive straightforward and clear responses from the knowledge base. However, for more specialized or urgent questions, the chatbot intelligently redirects the patient to consult a healthcare professional, ensuring that critical decisions remain in the hands of medical experts.

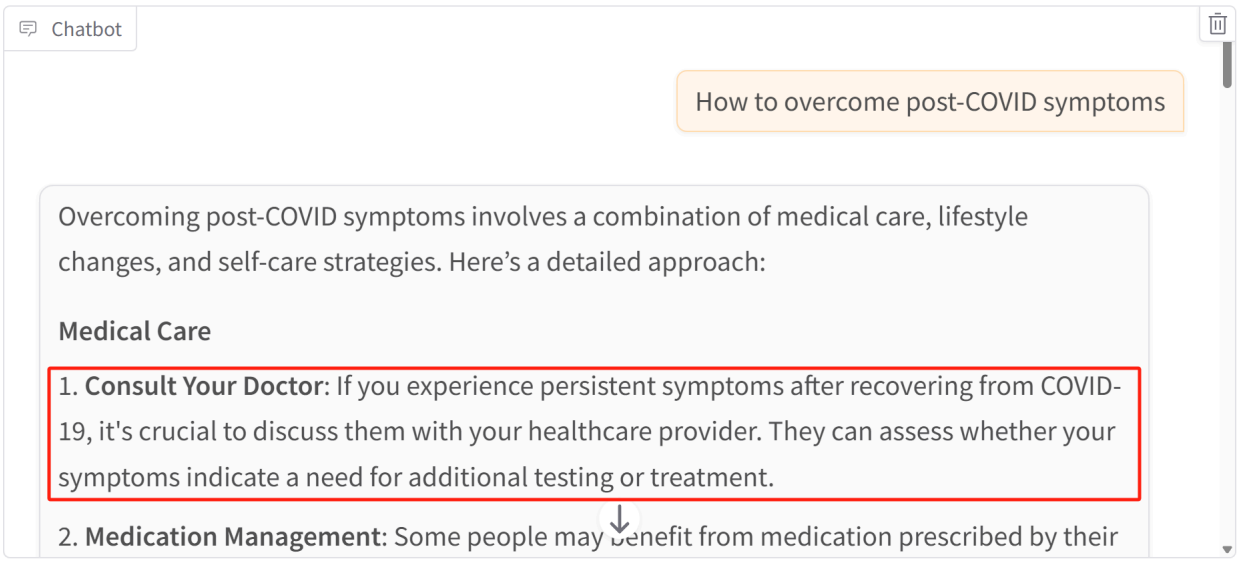


Figure 15: Chatbot categorize patient queries based on complexity

5.4.2 Chatbot Performance Testing (author: Yifan Wang)

Performance testing ensures that the system meets the required standards for resource utilization when the system is subjected to simulated workloads that mimic real-world usage patterns. High performance is critical, especially in emergency department settings where delays can significantly impact patient care. The team used NVIDIA GeForce RTX 4060, operating at a GPU utilization of 37% [Figure 16], which indicates that the chatbot can effectively operate within resource constraints while maintaining high-speed response generation [11].

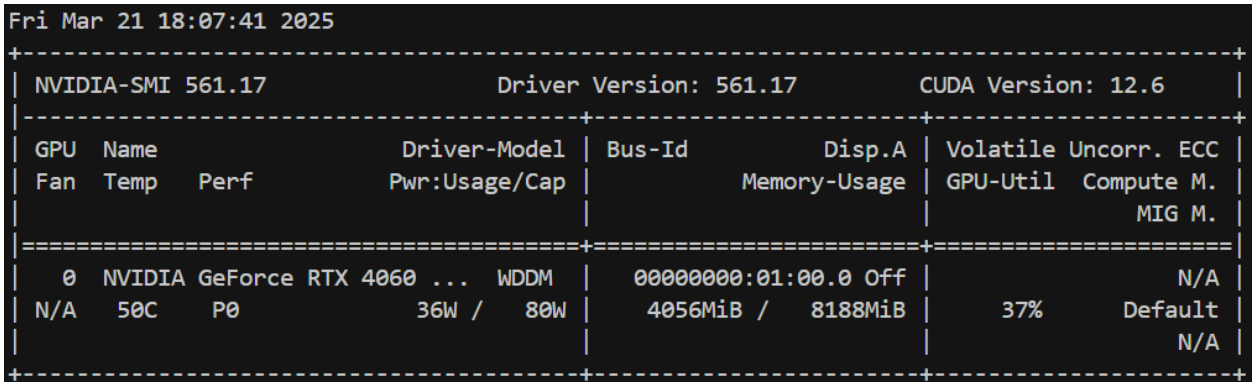


Figure 16: Chatbot Performance Testing on RTX 4060

5.4.3 Chatbot Testing Table (author: Yifan Wang)

Table 4: Chatbot Testing Table

ID	Requirement	Method	Result and Proof
5.1	Chatbot: proper interaction	Ask the chatbot for common health-related problems	PASS, See section 5.5.1 and Figure 11.
5.2	Chatbot: categorize patient queries based on complexity	Response prepared sample questions	PASS, successfully prepared sample questions.
5.3	Chatbot: normal resource usage	Test: less than 90% GPU-Util [11]	PASS, See section 5.5.2 and Figure 15.

6. Summary and Conclusions (author: Yuan Sui)

This project successfully created an intelligent hospital scheduling system with a user-friendly front end integrated into the back end, based on a Deep Q-Network, for the optimization of emergency departments (ED). This work has focused on developing a user-friendly job scheduler that aims at three specific objectives in segmentation: patient waiting times, resource utilization, and the real-time prioritization of high-severity cases. The project met its functional requirements, as proved through validation and acceptance tests, which indicated considerable advances over traditional rule-based scheduling methods.

The front-end, built using HTML and CSS, featured a main website, a unified login page, a patient chatbot, and a doctor interface, achieving webpage loading times under 1 second and button response times below 0.1 seconds. The back-end DQN model, trained on synthetic data reflecting real-world distributions, reduced weighted delays significantly compared to rule-based methods, as evidenced by the histogram showing lower average weighted waiting times across patient density levels 7 to 10, and the table indicating the DRL approach's lowest penalty patient percentages (e.g., 2.9% at density level 7). Systematic testing validated the system's functionality and usability, meeting the project's goals of enhancing scheduling efficiency and user experience. The design concepts, particularly the use of basic web technologies and reinforcement learning, were substantiated through these results, though the back-end's reliance on simulated data highlights a need for real-world validation.

The key takeaways are that an intuitive design combined with DQN-based scheduling can solve many ED challenges, as testing shows significant delays reduced. As live data integration, such as electronic health records and sensory information, was not done, the tested system does not represent the ability to adapt to dynamic real-time conditions. Further development should include multi-modal data stream integration and testing in actual hospital settings to further develop its robustness. This goes to show that the potential of reinforcement learning for scheduling in health care is immense; however, much of that dynamic, information-rich environment remains to be explored. It

would be reasonable to recommend real-world pilot deployments within hospitals to gather feedback, as hybrid modeling would further ensure responsiveness and predictive accuracy across a wider range of contexts in healthcare.

7. Future Work (author: Yifan Wang)

Future works in smart emergency department job scheduling will include the integration of multi-modal streams of data and testing advanced algorithmic strategies to enhance system flexibility and robustness. One such way is the use of real-time sensory data from hospital systems, full electronic health records (EHRs) [12], and continuous patient feedback to measure the full spectrum of drivers of patient flow. Through integrating these heterogeneous data streams, the system is able to construct more detailed images of dynamic clinical environments and thereby make more intelligent and context-aware scheduling decisions by the deployed Deep Reinforcement Learning (DRL) models. Additional future work must explore other DRL architectures and cross-paradigm hybrid models that take the best of supervised learning and reinforcement learning to push predictive accuracy and responsiveness under uncertainty even further. For instance, such models as recurrent neural networks (RNN) [13], or even transformer models can be employed to detect temporal trends and outliers in patient arrivals, resource utilization, and case severity, thus improving the performance of the scheduler in real time.

To continue the project, we recommend a systematic roadmap that starts with strictly controlled settings, where the model could first be stressed-test against variable conditions such as peak patient throughput, fluctuating resource availability, and sudden surges in demand for emergency services. It then needs to ground conditions for key performance indicators, such as decreased patient waiting times, enhanced efficiency of resource use, and increased patient satisfaction. After successful simulation testing, it should undergo a phased deployment within several hospitals. Such piloting is crucial for obtaining invaluable real-life operational feedback to make iterative refinements in the model in DRL and in the system's architecture as a whole. At this stage, critical attention should be paid to monitoring the developed system's performance metrics,

with further fine-tuning needed to assure that the system can deliver the clinical and operational functionalities required. In addition, because the proposed solution needs to be scalable, it must be tested within a wide range of healthcare settings, from small community hospitals to large metropolitan medical centers, which will prove its relative adaptability and effectiveness across diverse operating environments.

8. References (author: Ruoyu Li)

[1] M. Sartini et al., “Overcrowding in emergency department: Causes, consequences, and solutions-A narrative review,” Healthcare (Basel, Switzerland), https://pmc.ncbi.nlm.nih.gov/articles/PMC9498666/ (accessed Mar. 21, 2025).

[2] The Canadian triage and Acuity Scale: A Canadian perspective on ..., http://onlinelibrary.wiley.com/doi/10.1046/j.1442-2026.2003.00400.x/abstract?deniedAccessCustomisedMessage=&systemMessage=Wiley+Online+Library+will+be+disrupted+Saturday%2C+15+March+from+10%3A00-12%3A00+GMT+%2806%3A00-08%3A00+EDT%29+for+essential+maintenance&userIsAuthenticated=false (accessed Mar. 22, 2025).

[3] Ouhmidou, Ichraq & Bait Kulaib, Ghaida & Summad, Emad. (2020). Modelling and Optimizing Hospital Emergency Department Workflow.

[4] S. Di Somma, L. Paladino, L. Vaughan, I. Lalle, L. Magrini, and M. Magnanti, “Overcrowding in emergency department: an international issue,” *Internal and Emergency Medicine*, vol. 10, no. 2, pp. 171–175, Dec. 2014, doi: https://doi.org/10.1007/s11739-014-1154-8.

[5] “IMPROVED SCHEDULING OF OPERATING ROOMS - ProQuest,” *Proquest.com*, 2017. https://www.proquest.com/openview/ebf3ed5c9eac5e065ce39cce2713afb6/1?cbl=2037614&pq-origsite=gscholar (accessed Mar. 22, 2025).

[6] A. A. El Adoly, M. Gheith, and M. Nashat Fors, “A new formulation and solution for the nurse scheduling problem: A case study in Egypt,” *Alexandria Engineering Journal*, vol. 57, no. 4, pp. 2289–2298, Dec. 2018, doi: https://doi.org/10.1016/j.aej.2017.09.007.

[7] C. Duguay and F. Chetouane, “Modeling and Improving Emergency Department Systems using Discrete Event Simulation,” *SIMULATION*, vol. 83, no. 4, pp. 311–320, Apr. 2007, doi: https://doi.org/10.1177/0037549707083111.

[8] A. Roy, J. L. Pachuau, and A. K. Saha, “An overview of queuing delay and various delay based algorithms in networks - computing,” SpringerLink, https://link.springer.com/article/10.1007/s00607-021-00973-3 (accessed Mar.21, 2024).

[9] Health Quality Ontario, “Emergency Department Time Spent by Patients in Ontario – Health Quality Ontario (HQO),” *www.hqontario.ca*, 2023. https://www.hqontario.ca/System-Performance/Time-Spent-in-Emergency-Departments

[10] J. Nielsen, “Website response times,” Nielsen Norman Group, https://www.nngroup.com/articles/website-response-times/ (accessed Mar. 21, 2025).

[11] “The Ultimate Guide to GeForce 4060 & 4060 Ti Graphics Cards,” *NVIDIA*. https://www.nvidia.com/en-us/geforce/news/ultimate-guide-to-4060/

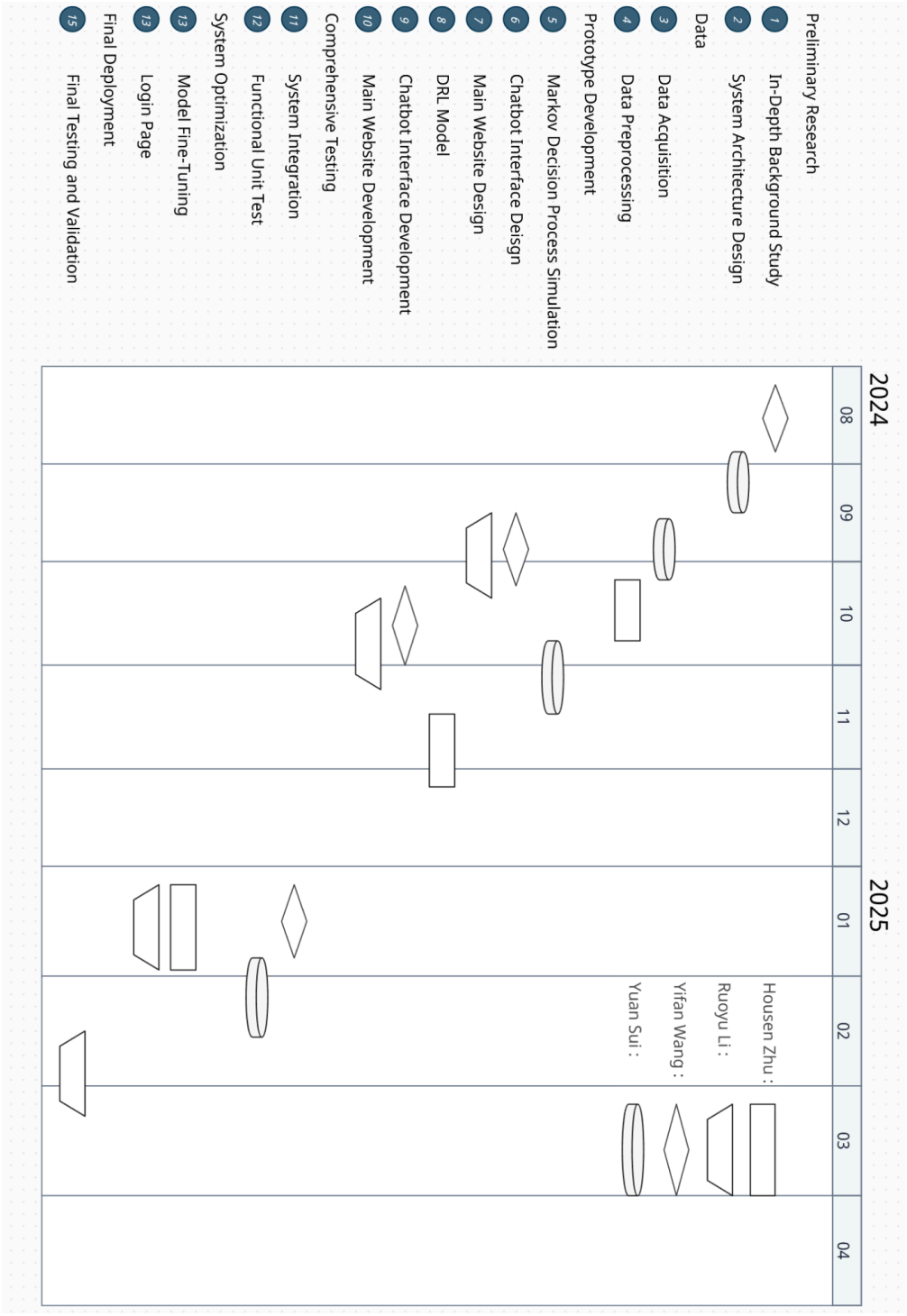
[12] “What’s an Ehr?,” eHealth Ontario, https://ehealthontario.on.ca/en/patients-and-families/ehrs-explained (accessed Mar. 21, 2025).

[13] A. Dabas, “Application of recurrent neural networks (rnns) in Medical Diagnostics,” Accueil - Archive ouverte HAL, https://hal.science/hal-04670386v1 (accessed Mar. 21, 2025).

[14] S. Lee and Y. H. Lee, “Improving Emergency Department Efficiency by Patient Scheduling Using Deep Reinforcement Learning,” *Healthcare*, vol. 8, no. 2, p. 77, Mar. 2020, doi: https://doi.org/10.3390/healthcare8020077.

9. Appendices (author: Housen Zhu)

A: Gantt chart History Appendix



B: Financial plan (updated from Implementation Plan)

Consumables & Services

Item	Priority	Cost/unit	Quantity (# of hours)	Total Cost	Requires Funding
Website	1	$52	3 years	$154	y
Colab membership	1	$14	4*4 months	$224	y
Total Consumables / Service				$378	
Total Requiring Funding				$378	

Funding	
Students ($100 ea)	$378
Total Funding	$378

Student Labour

Item	Cost/unit	Quantity (# of hours)	Total Cost
Housen Zhu	$7	600	$4200
Ruoyu Li	$7	600	$4200
Yifan Wang	$7	600	$4200
Yuan Sui	$7	600	$4200
Total Student Labour (unfunded)			$16800

Total Cost of Project	$17178
Total Cost Requiring Funding	$378

C: Complete training algorithm [14]

Input: Scheduling problem

Output: Parameters θ of a Q-Network

1: Initialize replay memory M

2: Initialize Q-Network with weights θ

3: Initialize target network Q^{-} with weights θ^{-}

4: for episode = 1, E do

5: for t = 1, T do

6: Patient arrival i with Poisson distribution

7: Get $\overline{\mathscr{K}}_{gt}$ and $\wp_{gt}$

8: if $\overline{\mathscr{K}}_{gt} \neq \emptyset$ and $\wp_{gt} \neq \emptyset$ then

9: repeat

10: Observe s_t

11: Select a random action a_t $(i \in \wp_{gt})$ with probability ε greedy policy. Otherwise, $a_t = \underset{a}{max} Q(s_t, a; \theta)$

12: Execute a_t and Observe r_t, s_{t+1}

13: Save transition (s_t, a_t, r_t, s_{t+1}) in M

14: $\overline{\mathscr{K}}_{gt} \leftarrow \overline{\mathscr{K}}_{gt} \setminus \{k\}$

15: $\wp_{gt} \leftarrow \wp_{gt} \setminus \{i\}$

16: until $\overline{\mathscr{K}}_{gt} = \emptyset$ or $\wp_{gt} = \emptyset$

17: end if

18: if $|M| \geq \frac{L}{2}$ and $t\%60 = 0$ then

19: Sampling random minibatch (s_h, a_h, r_h, s_{h+1}) from M

20: $y_h = r_h + \gamma \underset{a'}{max} Q(s_{h+1}, a'; \theta^{-})$

21: Perform a gradient decent regarding weights θ

22: end if

23: end for

24: Update $\theta^{-} = \theta$

25: end for

D: Rule-based algorithm used to compare with our deep reinforcement learning approach [14]

Dispatching rules.

Dispatching Rule	Description
First Come, First Served (FCFS)	Select the patient arriving at the earliest time in the ED
Shortest Remaining Processing Time (SRPT)	Select a patient having the shortest remaining average process time
Critical Ratio (CR)	Select a patient by dividing the actual time remaining for a particular treatment by the estimated time required for the entire treatment process
Acuity and SRPT (AS)	Select a patient based on the linear combination of two weighted variables: weighted acuity and SRPT
Acuity and Waiting Time (AW)	Select a patient based on the linear combination of two weighted variables: weighted acuity and weighted waiting time
Acuity and Arrival Time (AA)	Select a patient considering the high acuity level and earliest arrival time as primary and secondary factors, respectively

E: Emergency Department Simulator

